

1. 尝试通过不同方法对视频中搭建的 YOLOv1 模型进行调优，将完整的代码、训练日志、模型表现的分析报告（包含超参数调节的实验结果）进行整理，并分享在模型调优中的思考与经验。

Batch_size 从 16 一路降低到 2 才能跑得动，没时间做太多实验，后面补一下这个。

2. 假设检测的图像尺寸是 416*416，在 YOLOv3 中，最多可以检测多少物体？

在每个尺度的特征图上，每个网格单元（Grid Cell）会预测 3 个 边界框（3 boxes at each scale）

三个尺度分别是：

$13*13*3+26*26*3+52*52*3=10647$ 个边界框，故最多能检测 10647 个物体

- 3.锚框作用：

为模型提供一组预设的尺寸基准，让模型通过预测“偏移量”来代替预测“绝对坐标”，从而降低学习难度并大幅提升检测的召回率

- 4.YOLO 版本比较：

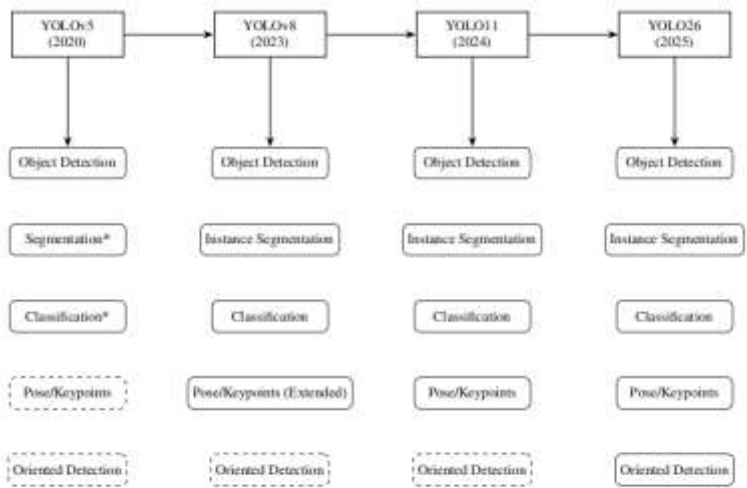


Figure 1: Timeline of Ultralytics YOLO models (YOLOv5, YOLOv8, YOLO11 and YOLO12) and their task support. Solid boxes = supported natively, dashed boxes = not supported. * indicates features added later via community extensions.

参考文献：arXiv:2510.09653v2

4.1 模型架构对比

- YOLOv5 (2020): PyTorch 模块化的奠基

架构基础：采用了基于 CSP（Cross Stage Partial）的主干网络和 PANet 风格的 Neck，以改善多尺度特征聚合。

激活函数：引入了 SiLU 激活函数，解决了深层网络中的梯度消失问题。

检测头与损失：使用传统的锚框（Anchor-based）机制和边界框回归，但在后期版本中

可选项 DFL (Distribution Focal Loss) 。

特点: 建立了模块化的 PyTorch 基础, 便于任务扩展和导出 。

- YOLOv8 (2023): 解耦头与无锚框设计

架构改进: 引入了 C2f 主干网络, 减少了参数量并保持了感受野的丰富性 。

检测头设计: 采用了解耦头 (Decoupled Head), 将分类和回归分支分开, 减少了优化过程中的干扰 。

预测机制: 正式采用**无锚框 (Anchor-free)**设计, 直接预测归一化的框参数, 简化了超参数调整 。

损失函数: 结合了 CIoU 和 DFL 损失以提高收敛性和小目标检测精度 。

- YOLO11 (2024): 效率与混合任务分配

特征提取: 引入了更紧凑的 C3k2 瓶颈层和 C2PSA 注意力模块, 在不增加延迟的情况下增强了空间注意力 。

任务分配: 采用了混合任务分配策略 (Hybrid Task Assignment), 联合调整分类、定位和辅助任务的损失权重 。

检测头: 虽然仍保留 NMS (非极大值抑制), 但通过更清晰的预测减少了后处理负载 。

- YOLO26 (2025): 端到端与去 DFL

架构简化: 移除了 DFL, 改用更轻量级、对硬件友好的边界框参数化方法 。

检测头创新: 实现了原生的无 NMS (NMS-free) 端到端推理, 消除了后处理延迟瓶颈 。

任务统一: 原生支持检测、实例分割、分类、姿态估计和旋转边界框检测五大任务 。

4.2 性能评估与分析

根据 MS COCO 数据集的基准测试, 各版本在精度与速度的权衡上表现出明显的代际差异。

YOLOv5: 作为基准, YOLOv5u-n 在 COCO 验证集上达到约 34.3% mAP, CPU 推理延迟约为 73.6 ms 。

YOLOv8: 相比 v5 有显著提升, YOLOv8n 达到 37.3% mAP, 但在 CPU 上的延迟略增至 80.4 ms 。

它在小目标检测上表现更好 。

YOLO11: 实现了效率的反超。YOLO11n 达到了 39.5% mAP, 且 CPU 延迟降低至 56.1 ms, 在同等延迟预算下比 v8 更准 。

YOLO26: 性能最强。YOLO26n 达到了 39.8% mAP（端到端模式下可达 40.3%），且 CPU 延迟大幅降低至 38.9 ms。

总结分析:

精度 (mAP): 每一代都在稳步提升，YOLO26l 版本达到了 53.0-53.4% mAP。

推理速度: YOLO26 通过移除 NMS 和 DFL，在 CPU 和嵌入式设备（如 Jetson）上实现了约 43% 的速度提升。

部署优势: YOLO26 的设计特别针对边缘计算进行了优化，消除了对 NMS 阈值调整的依赖。

4.3. 最新版本 YOLO26 的深入探讨

YOLO26 被定位为“边缘优先 (Edge-first)”的旗舰模型，其核心在于简化架构以换取极致的部署效率和稳定性。

A. 架构创新

移除 DFL (Distribution Focal Loss): 这一改变减少了计算图的复杂性，移除了在不同编译器和运行时中容易出错的算子，使得模型更容易量化。

原生无 NMS 推理: 解码路径被重构为端到端模式，直接输出紧凑且非冗余的预测结果。这不仅消除了传统的延迟瓶颈，还去除了部署时对 IoU 阈值等超参数的依赖。

B. 训练策略与损失函数

渐进式损失平衡 (ProgLoss - Progressive Loss Balancing):

YOLO26 引入了 ProgLoss，它可以动态调度分类、定位和辅助项的相对权重。

作用: 防止简单负样本或大目标在训练后期主导损失函数，从而提高收敛的平滑度。

小目标感知标签分配 (STAL - Small-Target-Aware Label Assignment):

原理: 调整分配先验和空间容忍度。

作用: 确保微小、被遮挡或低对比度的目标能获得足够的监督信号，显著提升了在无人机、智能相机等边缘场景下的召回率。

C. 优化算法

MuSGD 优化器:

这是一个混合优化器，结合了 SGD 的泛化能力和类似 Muon 优化器的曲率/动量感知更新（受大模型训练启发）。

效果: 缩短了达到高质量模型的时间，并减少了训练后期的震荡，有助于在不同模型

尺度（n/s/m/l/x）下实现可复现的收敛。

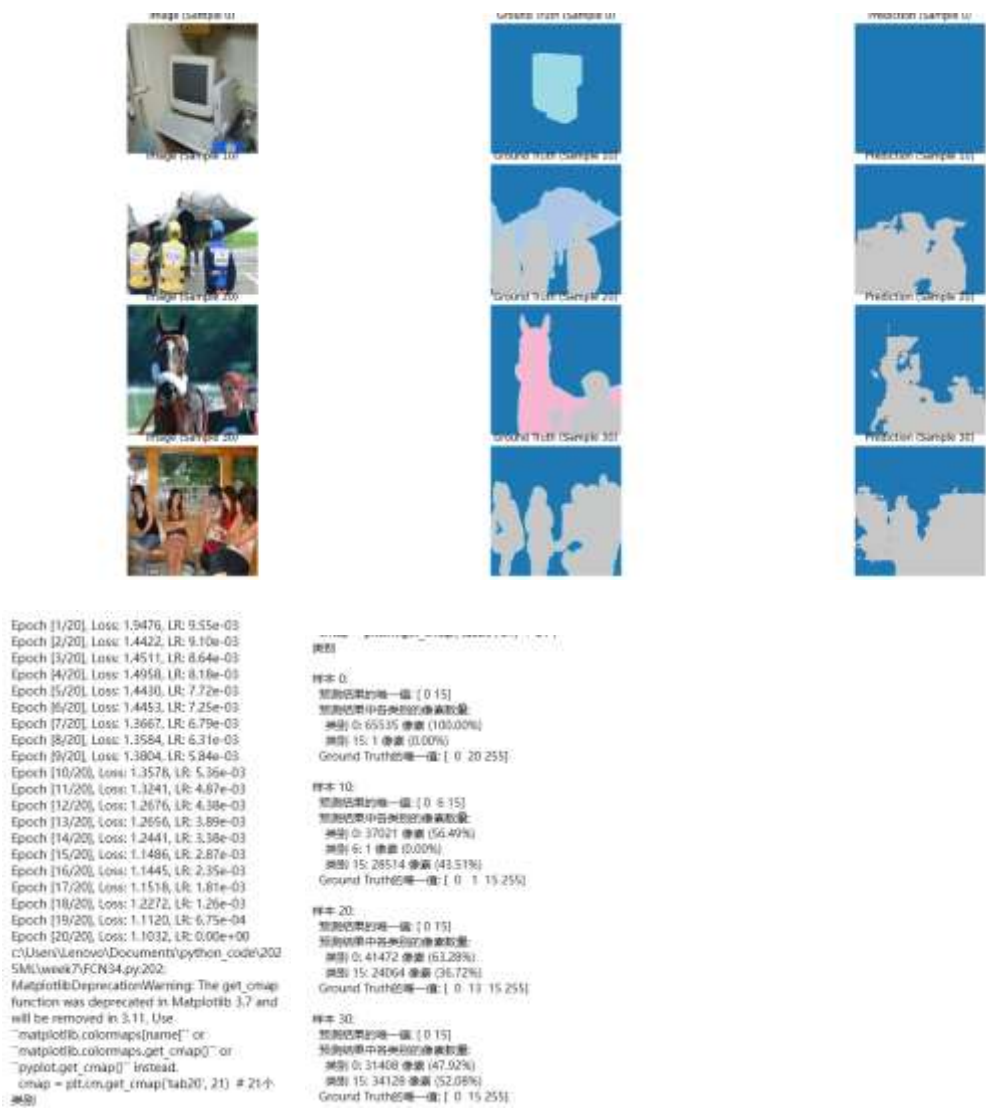
D. 部署与量化

量化鲁棒性: 由于移除了 DFL 和 NMS，YOLO26 的计算图避免了难以量化的自定义操作。

效果: 实验表明，YOLO26 的 INT8 导出模型几乎保留了与 FP32 版本相同的 mAP，非常适合在 NVIDIA Jetson Orin、高通 Snapdragon 等资源受限硬件上运行。

5.VOC 语义分割模型。

跑了两次



Run_1 问题:

- 视而不见" (Blindness to Minority Classes):

Sample 0 (Monitor): GT 有显示器 (Class 20), 预测全是背景。

Sample 10 (Plane): GT 有飞机 (Class 1), 预测全是人 (Class 15)。

Sample 20 (Horse): GT 有马 (Class 13), 预测全是人 (Class 15)。

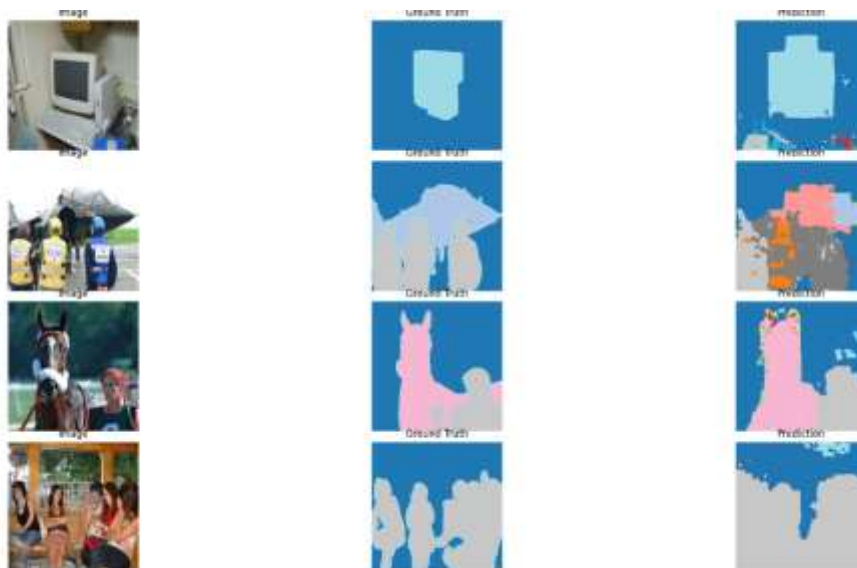
Sample 30 (People): GT 是人, 预测是人。这是唯一预测“正确”的, 因为模型学会了“遇事不决猜背景, 看到物体猜是人”的策略。

- Loss 仍然过高: 1.10 的 Loss 意味着模型对大多数像素的置信度依然很低, 或者在某些类上完全错误。

改进方法:

- ✓ 引入类别权重 (Class Weights): 在 CrossEntropyLoss 中, 将背景类 (Index 0) 权重设为 0.1, 其余 20 个物体类别权重设为 1.0。
- ✓ 采用混合损失函数 (Combined Loss): 新增 DiceLoss 类 (处理了 ignore_index=255), 训练时采用 Loss = CrossEntropy + DiceLoss。
- ✓ 增加训练轮数 (Epochs): 20 Epochs 提升至 50 Epochs

Run_2 效果:



```

使用优化策略: CrossEntropy(Background
Weight=0.1) + DiceLoss
Epoch [1/50] Total Loss: 3.7318 (CE: 2.7575,
Dice: 0.9743) LR: 9.82e-03
Epoch [2/50] Total Loss: 3.6136 (CE: 2.6454,
Dice: 0.9682) LR: 9.64e-03
Epoch [3/50] Total Loss: 3.5228 (CE: 2.5571,
Dice: 0.9657) LR: 9.46e-03
Epoch [4/50] Total Loss: 3.5091 (CE: 2.5448,
Dice: 0.9643) LR: 9.28e-03
Epoch [5/50] Total Loss: 3.3894 (CE: 2.4300,
Dice: 0.9593) LR: 9.10e-03
Epoch [6/50] Total Loss: 3.3535 (CE: 2.3947,
Dice: 0.9587) LR: 8.91e-03
Epoch [7/50] Total Loss: 3.3263 (CE: 2.3680,
Dice: 0.9583) LR: 8.73e-03
Epoch [8/50] Total Loss: 3.0932 (CE: 2.1453,
Dice: 0.9480) LR: 8.55e-03
Epoch [9/50] Total Loss: 3.0646 (CE: 2.1280,
Dice: 0.9367) LR: 8.36e-03
Epoch [10/50] Total Loss: 2.9005 (CE: 1.9718,
Dice: 0.9288) LR: 8.18e-03
Epoch [11/50] Total Loss: 2.9057 (CE: 1.9764,
Dice: 0.9293) LR: 8.00e-03
Epoch [12/50] Total Loss: 2.8526 (CE: 1.9316,
Dice: 0.9210) LR: 7.81e-03
Epoch [13/50] Total Loss: 2.8140 (CE: 1.8937,
Dice: 0.9203) LR: 7.63e-03
Epoch [14/50] Total Loss: 2.7107 (CE: 1.7989,
Dice: 0.9118) LR: 7.44e-03
Epoch [15/50] Total Loss: 2.5671 (CE: 1.6609,
Dice: 0.9062) LR: 7.25e-03
Epoch [16/50] Total Loss: 2.5146 (CE: 1.6103,
Dice: 0.9043) LR: 7.07e-03
Epoch [17/50] Total Loss: 2.4520 (CE: 1.5546,
Dice: 0.8994) LR: 6.89e-03

```

```

Epoch [46/50] Total Loss: 1.5027 (CE: 0.7157,
Dice: 0.7871) LR: 1.03e-03
Epoch [47/50] Total Loss: 1.4040 (CE: 0.6273,
Dice: 0.7767) LR: 7.95e-04
Epoch [48/50] Total Loss: 1.4735 (CE: 0.6893,
Dice: 0.7841) LR: 5.52e-04
Epoch [49/50] Total Loss: 1.4950 (CE: 0.7204,
Dice: 0.7747) LR: 2.96e-04
Epoch [50/50] Total Loss: 1.4885 (CE: 0.7006,
Dice: 0.7879) LR: 0.00e+00
c:\Users\Lenovo\Documents\python_code\202
5ML\week7\FCN34v2.py:234:
MatplotlibDeprecationWarning: The get_cmap
function was deprecated in Matplotlib 3.7 and
will be removed in 3.11. Use
``matplotlib.colormaps[name]`` or
``matplotlib.colormaps.get_cmap()`` or
``pyplot.get_cmap()`` instead.
cmap = plt.cm.get_cmap('tab20', 21)

```

```

样本 0: GT类别: [ 0 20 255] -> 预测类别: [ 0 6
9 11 14 15 18 20]

```

```

样本 10: GT类别: [ 0 1 15 255] -> 预测类别: [ 0
1 2 4 5 6 7 14 15]

```

```

样本 20: GT类别: [ 0 13 15 255] -> 预测类别:
[ 0 5 10 11 13 14 15 17 20]

```

```

样本 30: GT类别: [ 0 15 255] -> 预测类别: [ 0 2
6 9 11 14 15 16 19]

```

虽然识别轮廓还是有点粗糙，但是起码类别出现了新的（预测类别的唯一值列表中，样本 0 出现了 Class 20（Monitor），样本 10 出现了 Class 1（Aeroplane），样本 20 出现了 Class 13（Horse）），证明模仿 U-net 设置的类别权重（Class Weights）发挥了作用，打破了模型对背景的依赖。

6.为何 Unet 卷积不填充。

主要是避免边界伪影（Border Artifacts）使用填充的 3*3 卷积虽然可以保持图像尺寸不变，但会在图像边缘引入非真实的零值信息，这可能会干扰边缘处的分割精度。生物医学图像分割，对边缘信息非常敏感。

7.偶数

为了配合网络下采样（Downsampling）和上采样（Upsampling）结构稳定性的强制设计。确保 2x2 Max-pooling 的有效性，如果是奇数，会因为向下取整而导致信息不对齐。

另外，在扩张路径中，网络通过 2*2 的反卷积（Up-convolution）将尺寸扩大一倍。如果下采样时因为奇数尺寸丢失了边缘像素，上采样后就无法与收缩路径对应的特征图进行完美的尺寸匹配（即使经过裁剪），从而破坏了 U 型结构的对称性和拼接的准确性。

8.U-net 训练

- 激进的数据增强

对于生物细胞的显微镜图片的语义分割，采用了弹性形变（Elastic Deformations）的方式增强数据多样性，让网络学习到这种生物细胞的弹性不变性。

其他变换比较常规了，如旋转、平移、改变灰度等

- 加权损失函数

这个课程中已经详细讲过了，权重图在两个细胞接触的狭窄边界处具有很高的权重，迫使网络重点学习这些难以分割的像素

- 优化器策略

作者使用了 Caffe 的随机梯度下降（SGD），并设定了高达 0.99 的动量，确保了参数更新由大量之前看过的样本决定，从而稳定单样本训练的梯度。

- 高斯分布初始化

为了防止深层网络中的激活值爆炸或消失，初始权重从标准差为 $\sqrt{\frac{2}{N}}$ 的高斯分布中采样，N 是神经元输入节点数。